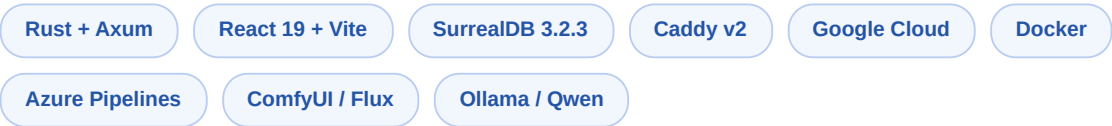


Fluency

Cómo está construida la plataforma: el stack tecnológico, la infraestructura multi-cloud (Google Cloud + estación local de generación + AWS + Azure) y el mapa de directorios del repositorio — explicado para poder leerse tanto por alguien sin experiencia en infraestructura como por un ingeniero que va a operarla.



Índice

1. Qué es Fluency, en una hoja
2. Cómo leer este documento
3. El mapa completo: de un clic del usuario a la base de datos
4. El stack tecnológico y por qué se eligió
5. Arquitectura de código: monolito modular "por pisos"
6. Los cinco lugares donde corre Fluency (infraestructura física)
7. La estación local (LocalBuild): dónde nacen las imágenes y el audio con IA
8. La base de datos: SurrealDB
9. Entrega de imágenes y audio: Cloudflare, Caddy y el versionado
10. Cómo se despliega: el pipeline de Azure
11. Resiliencia del proxy: overflow automático y tuning de sistema operativo
12. Dominios y rutas
13. Mapa de directorios del repositorio
14. Operación del día a día (comandos)
15. Glosario para quien no tiene experiencia en infraestructura
16. Dónde seguir leyendo (documentos vivos)

1. Qué es Fluency, en una hoja

Fluency (fluency.lat) es una plataforma web de aprendizaje de idiomas. El producto principal son **flashcards con imagen y audio generados por inteligencia artificial**, más un módulo de **práctica de pronombres** con historias interactivas y un tutor de gramática. Todo corre detrás de login con Google.

Técnicamente son dos piezas que se comunican por HTTP/JSON:

- Un **backend** escrito en Rust que expone una API REST (autenticación, progreso del usuario, generación y entrega de imágenes/audio, tutor de IA).
- Un **frontend** en React que es una SPA (aplicación de una sola página) servida como archivos estáticos: HTML/JS/CSS que corren en el navegador del usuario y hablan con la API.

Ambas piezas están organizadas como un **monolito modular**: no son microservicios separados, son un solo programa (o una sola SPA) que puede activar o desactivar "módulos de negocio" (flashcards, pronombres, panel de precios, panel admin) sin romper el resto — ver §5.

PARA QUIEN NO TIENE IDEA DE INFRAESTRUCTURA

Piensa en Fluency como un restaurante con dos partes: la **cocina** (backend Rust) prepara los pedidos (datos, imágenes, audio) y el **salón** (frontend React) es lo que el cliente ve y toca en su navegador. Ambos están en un edificio (los servidores) que alguien tiene que mantener encendido, con luz (electricidad/CPU/RAM) y agua (red/ancho de banda). Ese edificio hoy es, principalmente, una máquina virtual en **Google Cloud**.

2. Cómo leer este documento

El documento está escrito en capas: cada sección empieza con una explicación en lenguaje llano y luego baja al detalle técnico (rutas de archivos, variables de entorno, comandos). Vas a encontrar tres tipos de recuadro:

PARA QUIEN NO TIENE IDEA DE INFRAESTRUCTURA

Una analogía o explicación sin jerga, para entender el concepto antes de ver la tabla técnica.

PARA EL INGENIERO QUE VA A OPERAR ESTO

El detalle concreto: rutas reales, comandos, nombres de variables, límites de recursos.

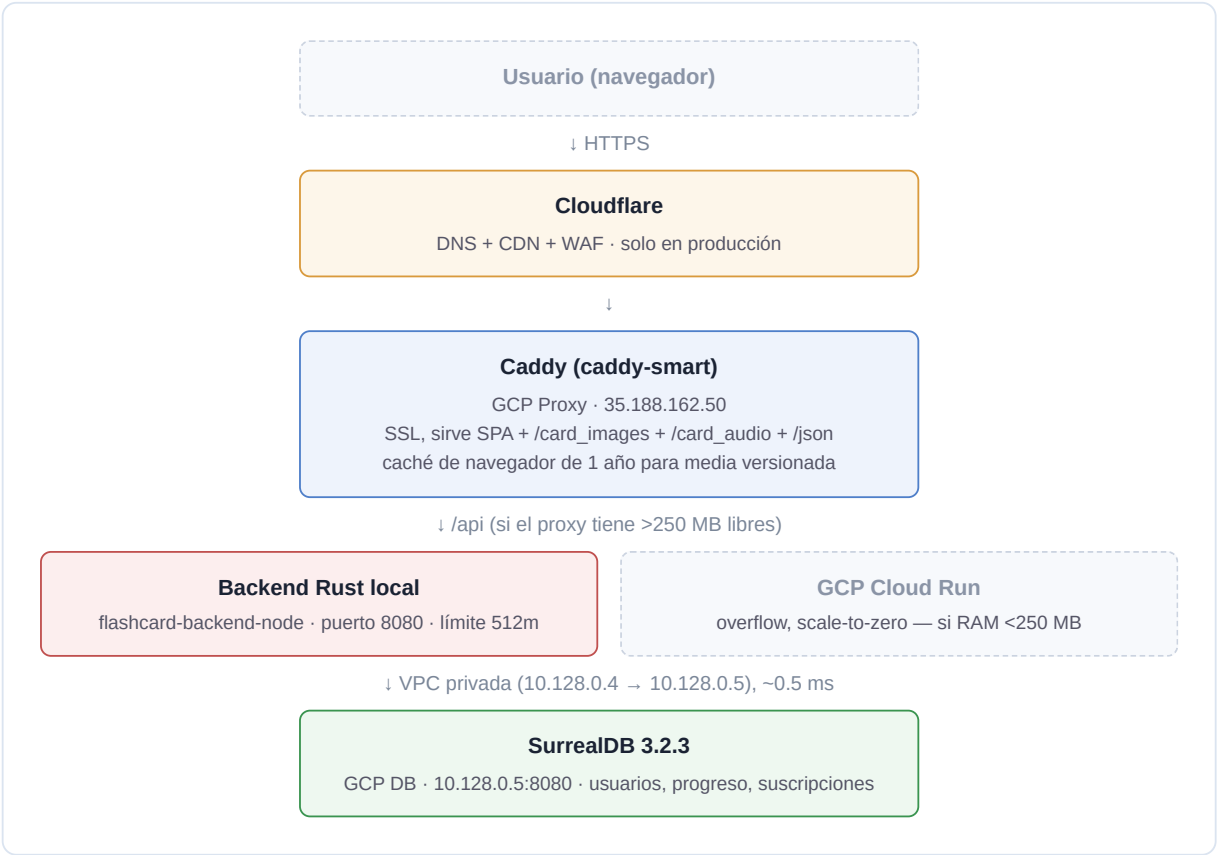
CUIDADO / INVARIANTE

Algo que si se rompe causa una falla real ya vivida en este proyecto (documentada en el repo).

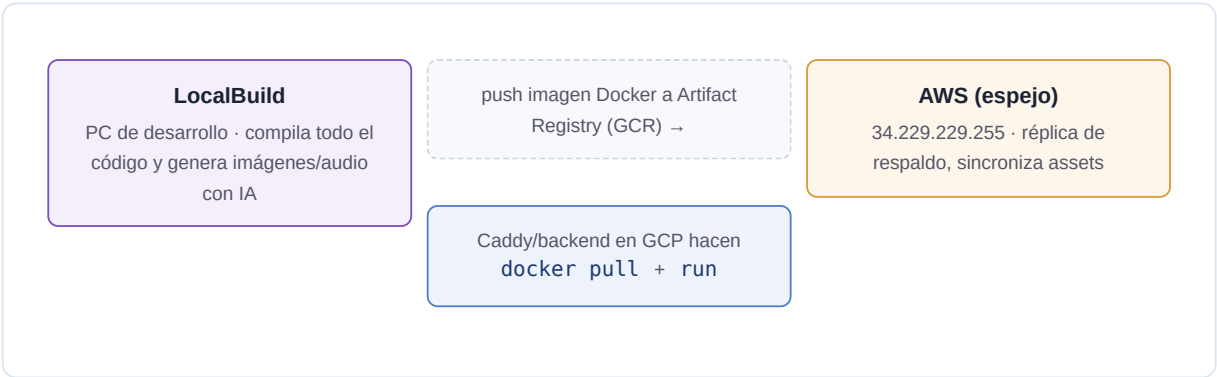
Si solo tienes 5 minutos: lee §1, §3 y la tabla de §6. Si vas a operar infraestructura o pipeline: lee además §7, §9, §10 y §11 completas antes de tocar nada en producción.

3. El mapa completo: de un clic del usuario a la base de datos

Así viaja una petición real en producción, hoy:



Al mismo tiempo, dos piezas más alimentan ese sistema pero no están en el camino de cada petición del usuario:



Lectura de esta figura, en palabras: el usuario nunca toca el PC de desarrollo ni a AWS directamente. El PC de desarrollo (*LocalBuild*) es quien **compila** el backend/frontend y quien **genera** las imágenes y el audio con modelos de IA locales; después esos artefactos (código empaquetado en una imagen Docker, o archivos de imagen/audio) se copian hacia el servidor real de Google Cloud, que es el único que atiende tráfico público. AWS es un espejo/respaldo.

INVARIANTE

Las dos VMs de Google Cloud (proxy y base de datos) son máquinas separadas de ~1 y ~2 GB de RAM respectivamente. No comparten memoria: mover trabajo de una a otra cambia el riesgo, no lo elimina. Ninguna de las dos compila código ni genera imágenes/audio — eso es exclusivo de LocalBuild.

4. El stack tecnológico y por qué se eligió

CAPA	TECNOLOGÍA	QUÉ HACE / POR QUÉ ESTA ELECCIÓN	DÓNDE VIVE EN EL REPO
Backend	Rust + Axum (sobre Tokio, async)	API HTTP. Rust se eligió por bajísimo consumo de RAM (~20 MB en reposo), sin recolector de basura y errores atrapados en tiempo de compilación — crítico porque los servidores de producción tienen 1–2 GB de RAM.	<code>backend/</code>
Frontend	React 19 + Vite	SPA servida como archivos estáticos. Vite compila/empaqueta rápido; React con un <i>registry</i> de módulos propio (no un framework de módulos de terceros).	<code>client/</code>
Base de datos	SurrealDB 3.2.3 (motor RocksDB embebido)	Base NoSQL orientada a documentos/grrafo. Guarda usuarios, progreso de flashcards, suscripciones y el contenido narrativo del módulo de pronombres. Los pagos (LemonSqueezy) también persisten aquí.	<code>backend/api_main/src/infrastructure/storage/surreal/</code>
Proxy / SSL / archivos estáticos	Caddy v2	Termina TLS (certificados Let's Encrypt automáticos), sirve el build de React, sirve <code>/card_images</code> y <code>/card_audio</code> directo desde disco, reenvía <code>/api</code> al backend (con overflow — ver §11).	<code>infra/proxy/Caddyfile.gcp</code>
CDN / seguridad de borde	Cloudflare	DNS autoritativo, proxy naranja en producción (oculta la IP real, filtra bots/WAF) y una regla de caché específica para imágenes/audio versionados.	configurado en el panel de Cloudflare; variable <code>MEDIA_DELIVERY_MODE=cloudflare</code>
Autenticación	Google OAuth 2.0 → JWT	Login con Google; el backend emite su propio JWT firmado para las peticiones siguientes.	<code>backend/mod_shell/src/auth.rs</code> , puerto <code>TokenVerifier</code>
Tutor de IA / explicaciones	Google Gemini	Corrige y explica errores gramaticales del usuario en el módulo de pronombres y en el tutor.	<code>backend/api_main/src/infrastructure/ai/</code>
Audio (texto → voz)	Gemini TTS (producción); ElevenLabs (solo demo de landing)	Genera el audio de cada frase de flashcard bajo demanda, cuando no existe ya en disco.	<code>mod_flashcards/src/batch/audio.rs</code> + adaptadores en <code>infrastructure/ai/</code>
Imágenes con IA	ComfyUI + Flux (generación) · Ollama + Qwen (refinado de prompt)	Corren <i>localmente</i> en el PC de desarrollo, cada uno en su propia GPU (ver §7) — no en la nube, por costo y porque necesitan GPU con VRAM que las VMs de 1–2 GB no tienen.	<code>mod_flashcards/src/batch/</code> (CLI batch), servicios systemd en LocalBuild

Contenedores	Docker	Backend, Caddy y SurrealDB corren cada uno en su propio contenedor en las VMs de producción.	<code>backend/Dockerfile</code> , <code>infra/proxy/Dockerfile</code>
CI/CD	Azure Pipelines (Azure DevOps)	Compila (en un agente self-hosted que es el propio PC de desarrollo) y despliega (en un segundo agente self-hosted) — ver §10.	<code>azure-pipelines.yml</code>
Pagos / suscripciones	LemonSqueezy (checkout hospedado + webhooks), estado sobre SurrealDB	Integración real y activa en el código , no un plan a futuro: <code>/pricing</code> y <code>/checkout</code> piden al backend una sesión de checkout que redirige a LemonSqueezy; el pago se confirma de forma asíncrona vía <code>POST /api/webhooks/lemonsqueezy</code> , que activa la suscripción en <code>subscription</code> de SurrealDB — no usa Postgres.	<code>backend/api_main/src/infrastructure/payment/ (lemonsqueezy_provider.rs)</code> , <code>client/src/modules/pricing/</code>

5. Arquitectura de código: monolito modular "por pisos"

PARA QUIEN NO TIENE IDEA DE INFRAESTRUCTURA

Imagina un edificio de oficinas. Hay un **lobby y ascensores compartidos** (login, layout, notificaciones — esto se llama el *"shell"*) y luego cada **piso** es un módulo de negocio independiente: flashcards, práctica de pronombres, panel de precios, panel admin. Se puede "clausurar un piso" (desactivar un módulo) sin que el edificio se caiga, y se puede vender solo algunos pisos a un cliente sin arrastrar el resto.

En términos de código esto se llama **arquitectura hexagonal / Clean Architecture**, aplicada de forma simétrica en backend y frontend: la lógica de negocio (*use cases*) nunca depende directamente de una tecnología concreta (base de datos, HTTP, proveedor de IA); depende de un **contrato/puerto**, y quien elige la implementación concreta (*adapter*) es el punto único de ensamblado (*composition root*).

5.1 Backend — capas

CAPA	DIRECTORIO	RESPONSABILIDAD
Dominio + puertos (contratos)	<code>backend/core/</code> (crate <code>fluency_core</code>)	Modelos de datos y traits/contratos: <code>StorageRepository</code> , <code>MediaDeliveryProvider</code> , <code>AITutor</code> , <code>TokenVerifier</code> , etc. No sabe nada de Rust web, HTTP ni SurrealDB.
Aplicación (casos de uso)	<code>backend/mod_shell/</code> , <code>backend/mod_flashcards/</code> , <code>backend/mod_pronoun/</code>	La lógica de negocio de cada módulo: auth, presencia, decks de flashcards, generación de imagen/audio, historias de pronombres. Cada módulo es su propio <i>crate</i> de Rust.
API HTTP	<code>backend/api_main/src/api/</code>	Handlers delgados: reciben el request, mapean DTOs → caso de uso, devuelven respuesta. No contienen lógica de negocio.
Infraestructura (adaptadores)	<code>backend/api_main/src/infrastructure/</code>	Implementaciones concretas por puerto: SurrealDB (<code>storage/surreal/</code>), entrega media Cloudflare/directa (<code>media_delivery/</code>), Gemini (<code>ai/</code>), OAuth (<code>auth/</code>), geolocalización (<code>geo/</code>), pagos (<code>payment/</code>).
Composition root	<code>backend/api_main/src/main.rs</code>	El único lugar que decide "qué adaptador real usar para cada puerto" y arma el estado compartido de la aplicación (<code>AppState</code>).
Registro de rutas por módulo	<code>backend/api_main/src/modules/</code>	Cada módulo activo registra sus rutas HTTP aquí (<code>register_routes()</code>).

PARA EL INGENIERO

Los módulos se activan/desactivan por **Cargo features** (`backend/api_main/Cargo.toml`):

```
cargo build -p api_main --no-default-features --features auth,flashcards # solo flashcards
cargo build -p api_main --no-default-features --features auth,pronoun_practice # solo pronombres
```

Si SurrealDB no está disponible, se inyecta `NullDbRepository` (patrón Null Object) y la app arranca igual, degradada (sin auth persistente). Además, la conexión inicial reintenta de forma acotada (6 intentos, ~10s cada uno) antes de degradar — ver `connect_surreal_with_retry` en `main.rs`.

5.2 Frontend — capas

El frontend replica el mismo patrón: `ports/` (contratos) → `useCases/` o `queries/` (lógica) → `adapters/` (HTTP concreto) → `composition.js` (ensamblado) → `index.jsx` (manifiesto que el *registry* descubre automáticamente).

CARPETA DENTRO DE UN MÓDULO	ROL
<code>ports/</code>	Contratos de datos/servicios (equivalente a los <i>traits</i> de <code>fluency_core</code>)
<code>adapters/</code>	Llamadas HTTP reales, usando siempre <code>client/src/services/httpClient.js</code>
<code>useCases/</code> / <code>queries/</code>	Lógica de aplicación pura, testeable sin React
<code>hooks/</code>	Orquestación React sobre esos casos de uso
<code>composition.js</code>	Inyección de dependencias del módulo (equivalente a <code>main.rs</code>)
<code>context/</code>	Estado React exclusivo del módulo (nunca en el <code>UIContext</code> del shell)
<code>config/</code>	Configuración/i18n exclusiva del módulo
<code>index.jsx</code>	Manifiesto: rutas, secciones de menú, overlays — lo que el <i>registry</i> global lee

El *registry* (`client/src/modules/index.js`) descubre automáticamente qué módulos existen en disco y los activa según flags de entorno (`VITE_ENABLE_*`). El shell compartido (`App.jsx` , layout, `AuthContext`) nunca importa código de un módulo directamente.

5.3 Sparse-checkout: aislar físicamente lo que no se necesita

PARA QUIEN NO TIENE IDEA DE INFRAESTRUCTURA

`git sparse-checkout` es una función de Git que permite tener **solo una parte** de los archivos del repositorio descargados en el disco, aunque todos existan en el historial. Sirve para que, si estás trabajando solo en "flashcards", ni tú ni una IA asistente vean por accidente el código de "pronombres" — reduce confusión y contexto innecesario.

```
./scripts/sparse-module.sh status      # solo lectura, siempre permitido
./scripts/sparse-module.sh flashcards  # activar un perfil (requiere autorización)
./scripts/sparse-module.sh full        # repo completo (requiere autorización)
```

INVARIANTE

Cambiar de perfil sparse cambia qué archivos *versionados* materializa Git; el repositorio de este documento (perfil activo hoy: `flashcards`) por eso no tiene en disco `backend/mod_pronoun/` ni `client/src/modules/pronounPractice/` — no significa que no existan en la arquitectura, solo que no están descargados en esta copia local. El plano sigue disponible en `docs/modules/pronoun.md` .

6. Los cinco lugares donde corre Fluency (infraestructura física)

Fluency es **multi-cloud**: no todo vive en un único proveedor. Esto es deliberado (evita depender de un solo proveedor) pero también es una fuente real de complejidad operativa — cada nodo tiene su propio rol y no hay "memoria compartida" entre ellos.

NODO	PROVEEDOR	ROL	SPECS	DIRECTORIOS CLAVE
fluency-proxy-backend GCP	Google Cloud, proyecto fluency	Punto de entrada real: Caddy (SSL, estáticos) + backend Rust de producción. Hace overflow a Cloud Run bajo presión de RAM (ver §11).	e2-micro, 2 vCPU, 1024 MB RAM + 4 GB swap , Alpine Linux (raíz es <i>tmpfs</i> diskless de ~485 MB — el disco persistente es /mnt/sda)	/mnt/sda/repository/flashcard/ (assets: json, card_images, card_audio), /mnt/sda/Caddyfile , /mnt/sda/caddy_data (certificados TLS)
fluency-db-surreal GCP	Google Cloud, proyecto fluency	Solo base de datos: únicamente SurrealDB, sin IP pública.	e2-small, 2 vCPU, 2048 MB RAM , Alpine Linux	volumen del contenedor SurrealDB (RocksDB); acceso solo por VPC privada 10.128.0.5:8080
LocalBuild Local	PC de desarrollo (no es nube)	Compila TODO el código y genera TODA la media (imágenes/audio) por lotes — ver §7.	~30 GB RAM, GPU 0 (RTX 5060 Ti 16 GB) + GPU 1 (GTX 1660 6 GB)	~/Desktop/dev/flashcard (este repo), ~/Desktop/dev/ComfyUI
alpine-aws-01 AWS	Amazon Web Services, us-east-1	Espejo/respaldo: réplica de backend + assets, sincroniza hacia el proxy real.	t3.micro, 2 vCPU, 1024 MB RAM , Alpine Linux	túnel WireGuard privado hacia el proxy real para SCP de assets
worker-alpine-native-1 Azure	Microsoft Azure	Infraestructura auxiliar sin uso activo hoy. Se reservó en su momento para una futura capa de pagos/transacciones sobre Postgres, pero los pagos ya se implementaron por otra vía (LemonSqueezy + SurrealDB, ver §4) — este nodo no interviene en el checkout real.	2 vCPU ARM64, 1024 MB RAM , Alpine Linux	—

PARA EL INGENIERO

Además existe un **proyecto GCP distinto** ([launch-490115](#)) que aloja el Artifact Registry ([gcr.io/launch-490115/flashcard-backend](#) , donde se publican las imágenes Docker del backend) y un servicio de **Cloud Run** como overflow serverless (0–10 instancias, 512 MB–1 GB por instancia) — Cloud Run no es la vía principal, solo absorbe presión de RAM cuando el proxy la reporta baja. No confundir credenciales/proyecto entre [fluency](#) (VMs reales) y [launch-490115](#) (registry + overflow).

INVARIANTE DE RECURSOS

Los límites Docker (p. ej. backend de producción a `512m` , SurrealDB a `800m`) son **techos, no reservas**. Que la suma de techos supere la RAM física de la VM no implica que esa memoria esté asignada de forma permanente — pero tampoco se debe subir un límite "porque hay memoria libre" sin medir RSS/picos/swap primero. Nunca compilar (`cargo build` , `docker buildx` , `vite build`) ni generar catálogos completos de media en las VMs de 1–2 GB: eso es exclusivo de LocalBuild.

7. La estación local (LocalBuild): dónde nacen las imágenes y el audio con IA

Este es el nodo que más se suele pasar por alto al pensar "infraestructura en la nube", pero es el más intensivo en cómputo de todo el sistema: **ningún servidor cloud de Fluency compila código ni genera imágenes/audio**. Todo ese trabajo pesado ocurre en el PC de desarrollo, llamado `LocalBuild` en el pipeline de CI/CD.

PARA QUIEN NO TIENE IDEA DE INFRAESTRUCTURA

Generar una imagen con IA o compilar un programa grande consume mucha memoria y, en el caso de imágenes, necesita una **tarjeta gráfica (GPU)** potente. Los servidores en la nube de Fluency son deliberadamente pequeños y baratos (1–2 GB de RAM, sin GPU) porque solo tienen que **servir** tráfico ya generado, no **generarlo**. El trabajo pesado se hace una sola vez, en una PC con GPU, y el resultado (archivos de imagen/audio ya listos) se copia a los servidores.

7.1 Qué corre en LocalBuild

FUNCIÓN	HERRAMIENTA	DETALLE
Generación de imágenes	ComfyUI + Flux	Servicio systemd <code>comfyui.service</code> , fijado a la GPU 0 (RTX 5060 Ti, 16 GB VRAM) con <code>CUDA_VISIBLE_DEVICES=0</code> , escucha en <code>127.0.0.1:8188</code> , flag <code>--cache-none</code> .
Refinado de prompts	Ollama + Qwen	Fijado a la GPU 1 (GTX 1660, 6 GB VRAM) vía override systemd con <code>CUDA_VISIBLE_DEVICES=1</code> , escucha en <code>127.0.0.1:11434</code> . Mejora el prompt antes de mandarlo a Flux.
Compilación frontend	Bun + Vite	Build de producción de la SPA React
Compilación backend	<code>docker buildx</code>	Cross-compile Rust para <code>linux/amd64</code> y <code>linux/arm64</code> a la vez, y <code>push</code> a Artifact Registry
Servicios de desarrollo	Backend Rust local, Vite dev server, SurrealDB local	Puertos <code>:8081</code> (backend dev), <code>:5173</code> (Vite), <code>:8001</code> (SurrealDB) — levantados por <code>./start.sh</code>

INVARIANTE

La separación de ComfyUI y Ollama en GPUs distintas es deliberada: juntarlos en la misma GPU causó errores de memoria (OOM) de PyTorch en el pasado. No revertir esa separación.

7.2 Cómo llega ese trabajo a producción

Hay dos caminos distintos, que conviene no confundir:

- Código** (backend/frontend): LocalBuild compila y sube una **imagen Docker** a Artifact Registry (`gcr.io/launch-490115/flashcard-backend:latest`); el pipeline de Azure hace que el servidor de Google Cloud haga `docker pull` + `docker run` de esa imagen. Esto pasa **en cada despliegue** (push a `main` / `qa` que toque `backend/`, `client/`, `infra/` o el propio pipeline).
- Media generada por lotes** (imágenes/audio para el catálogo completo de flashcards): se genera **manualmente, bajo demanda**, con el CLI batch del backend (`mod_flashcards/src/batch/`, invocado con flags `--batch-gen-images` / `--batch-gen-audio`) y después se sincroniza al repositorio real del proxy con scripts dedicados. **No** ocurre automáticamente en cada push.

El único contenido que sí viaja en *cada* despliegue automático es `json/` (el texto de las flashcards, sin imágenes ni audio) — y solo si se activa el parámetro `deployJson` del pipeline. Ver §10.

8. La base de datos: SurrealDB

PARA QUIEN NO TIENE IDEA DE INFRAESTRUCTURA

SurrealDB es una base de datos **NoSQL** orientada a documentos (parecida en espíritu a MongoDB, no a algo tipo Excel/tablas relacionales estrictas como Postgres o MySQL). Guarda "registros" con forma de JSON. **No es PostgreSQL** — esa confusión ya generó documentación incorrecta en el pasado dentro de este mismo proyecto.

Vive en su propia VM dedicada (`fluency-db-surreal` , sin IP pública, solo accesible por la red privada de Google Cloud) usando el motor de almacenamiento **RocksDB** embebido. El backend Rust se conecta por WebSocket a `10.128.0.5:8080` .

8.1 Tablas principales

TABLA	PARA QUÉ
<code>user</code>	Perfil de Google (email, nombre, foto) + rol (<code>viewer</code> , <code>premium</code> , <code>admin</code>)
<code>subscription</code>	Estado del plan (activo/cancelado/expirado), proveedor de pago
<code>card_progress</code>	Qué tarjetas aprendió cada usuario, más los campos opcionales de repaso espaciado (SRS) calculados en el cliente
<code>stories</code> , <code>episodes</code> , <code>story_screens</code>	Contenido narrativo del módulo de práctica de pronombres
<code>user_progress</code> , <code>user_errors</code>	Avance de cada jugador y bitácora de errores gramaticales corregidos por Gemini
<code>demo_feedback</code>	Reseñas que deja un usuario tras probar el demo público de la landing
<code>daily_stats</code> , <code>user_activity_stats</code>	Métricas agregadas de uso diario (usuarios activos, altas nuevas) y por usuario (racha, tiempo de estudio, país/dispositivo del último acceso)

Diagrama entidad-relación completo: `database_schema_diagram.md` en la raíz del repo.

INVARIANTE YA VIVIDO

El backend se conecta con `Surreal::new::<Ws>(endpoint)` , que espera la dirección **sin** esquema (`10.128.0.5:8080` , nunca `ws://10.128.0.5:8080`). Pasarle el esquema no da un error visible: la conexión se cuelga hasta timeout y la app degrada silenciosamente a modo sin base de datos (login roto, cualquier cuenta nueva aparece con el rol por defecto). Ver `backend/api_main/src/infrastructure/storage/surreal/connection.rs` .

9. Entrega de imágenes y audio: Cloudflare, Caddy y el versionado

PARA QUIEN NO TIENE IDEA DE INFRAESTRUCTURA

Cuando el navegador pide una imagen dos veces, no queremos volver a descargarla ni queremos que quede "pegada" para siempre si se reemplaza. La solución de este proyecto es simple: cada imagen se pide con un sufijo `?v=...` que cambia solo cuando el archivo realmente cambió. Mientras el sufijo sea el mismo, el navegador y Cloudflare pueden guardarla en caché con seguridad "para siempre"; en cuanto el archivo se reemplaza, el sufijo cambia y todo el mundo pide la versión nueva automáticamente — sin tener que "purgar" ni renombrar nada a mano.

El sistema puede operar en dos modos, controlados por una sola variable (`MEDIA_DELIVERY_MODE`):

MODO	CUÁNDO SE USA	CÓMO ENTREGA
<code>cloudflare</code>	Producción actual	Cloudflare cachea en su borde (edge) las URLs versionadas; el navegador revalida contra Cloudflare, no directamente contra el origen.
<code>oracle</code> (nombre legado en el código, hoy sirve igual desde GCP)	QA (<code>qa.fluency.lat</code> , DNS-only, sin Cloudflare) y valor de respaldo/rollback	Caddy sirve el archivo directo desde disco; el navegador puede cachear la URL versionada hasta un año como <code>immutable</code> .

El versionado se calcula así: `/card_images/tarjeta-123.avif?v=<fecha-modificación-nanosegundos>-<tamaño>` . No se regenera el catálogo, no se calcula un hash del contenido, no hay caché binaria en RAM del backend — solo se consultan metadatos del archivo (fecha + tamaño) al construir la URL.

PARA EL INGENIERO

Diseño hexagonal del puerto: `backend/core/src/ports/media_delivery.rs` define el contrato `MediaDeliveryProvider` ; los adaptadores concretos viven en `backend/api_main/src/infrastructure/media_delivery/` y se seleccionan en el *composition root*. Nunca hay un `if proveedor == "cloudflare"` dentro de un handler HTTP. En Caddy, la política de caché real (`Cache-Control` largo para media versionada, `no-cache` para la sin versión) vive en el snippet `asset_cache_policy` de `infra/proxy/Caddyfile.gcp` .

REGLA QUE NO SE DEBE ROMPER

La *cache key* de Cloudflare debe incluir la query completa. Activar `Ignore Query String` haría que versiones distintas (bytes distintos) se sirvieran como si fueran la misma — serviría contenido viejo aunque el archivo ya se haya reemplazado.

Detalle exhaustivo (headers exactos, procedimiento de panel de Cloudflare, pruebas): `docs/infrastructure/media-delivery-cache.md` .

10. Cómo se despliega: el pipeline de Azure

PARA QUIEN NO TIENE IDEA DE INFRAESTRUCTURA

Un "pipeline de CI/CD" es una receta automática que corre cada vez que alguien sube código nuevo: compila, empaqueta y lo instala en los servidores reales, en orden y sin intervención manual (salvo aprobar que corra). Aquí ese pipeline vive en Azure DevOps y tarda entre 25 y 45 minutos en completarse.

10.1 Dos "agentes" (máquinas que ejecutan el pipeline)

POOL	MÁQUINA REAL	HACE	NO HACE
LocalBuild	El PC de desarrollo (\$7)	Compila frontend (bun/vite) y hace cross-compile del backend (Rust, amd64+arm64) vía <code>docker buildx</code> , sube la imagen a Artifact Registry	Nunca toca servidores de producción directamente
Default	Agente self-hosted ARM	SSH/SCP a servidores, <code>docker pull</code> , <code>gcloud</code> , corre los scripts de <code>infra/proxy/</code>	Nunca compila Rust ni frontend

El agente `LocalBuild` debe estar encendido y online cuando corre el pipeline — si el PC está apagado, el pipeline falla en sus primeros dos stages.

10.2 Las etapas



Total esperado: ~25-45 minutos. Nunca encolar dos builds a la vez (manual + push automático) — compiten por el mismo agente `Default` y duplican tiempo/artefactos.

10.3 Disparo y secretos

- **Automático:** push a `main` o `qa` que toque `azure-pipelines.yml`, `client/**`, `backend/**` o `infra/**`.
- **Secretos:** viven en Azure DevOps (Variable Group `Flashcard-Secrets`), nunca en disco en el servidor destino; se pasan por SSH en modo `inline` (una sola sesión de shell, para que las variables `export` persistan entre comandos).

Detalle exhaustivo de cada stage, troubleshooting y patrones obsoletos a no reintroducir: `docs/infrastructure/pipeline-and-deploy.md`.

PARA EL INGENIERO

El despliegue del proxy real de Google Cloud (el que sirve `fluency.lat`) y del tuning de sistema operativo que describe §11 **todavía no están automatizados en este pipeline** — se hacen a mano por SSH, o corriendo `infra/proxy/bootstrap-gcp.sh` directamente en la VM. Es un candidato natural para un futuro stage 4b si se decide automatizarlo.

11. Resiliencia del proxy: overflow automático y tuning de sistema operativo

El proxy real (`fluency-proxy-backend`) es una máquina pequeña (1 GB de RAM). Para que un pico de tráfico no la colapse, tiene dos capas de defensa: una válvula que desvía tráfico cuando hace falta, y un sistema operativo afinado para exprimir mejor los recursos que sí tiene.

11.1 Válvula de overflow: API local → Cloud Run

PARA QUIEN NO TIENE IDEA DE INFRAESTRUCTURA

Un vigilante revisa cada 30 segundos cuánta memoria libre le queda a la máquina. Si hay suficiente, la propia máquina sigue atendiendo los pedidos de la API. Si empieza a quedarse sin memoria, en vez de colapsar, redirige esos pedidos a una copia idéntica del mismo programa que vive en **Google Cloud Run** — un servicio que solo se enciende cuando hace falta y no cuesta nada mientras está apagado.

Un proceso en segundo plano (`oracle-ram-monitor.sh` — nombre histórico del script, la lógica no cambió) revisa la RAM libre cada 30 segundos y gestiona un archivo de estado:

```
RAM libre > 250 MB → existe /tmp/ORACLE_HEALTHY → Caddy atiende con el Rust local
RAM libre ≤ 250 MB → no existe                      → Caddy desvía ese pedido a Cloud Run
```

Caddy decide esto **en cada pedido** a `/api/*` mirando si el archivo existe en ese instante — no es un interruptor único que se tira una vez, aunque en la práctica todos los pedidos de una ventana de ~30 s van al mismo lado. La respuesta incluye un header `X-Backend` (`GCP-Local` o `CloudRun-Overflow`) para poder distinguir cuál atendió cada petición.

CUIDADO

Cloud Run escala a cero instancias sin tráfico ("scale-to-zero"): el primer pedido que le llega después de un rato inactivo tiene un pequeño retraso ("cold start", unos segundos) mientras levanta una instancia nueva. Los siguientes, mientras siga con tráfico, son rápidos. Verificado en vivo: con la VM real bajo presión de RAM, el tráfico se desvió solo y siguió sirviendo datos reales sin caerse.

11.2 Tuning de sistema operativo

OPTIMIZACIÓN	QUÉ HACE	DÓNDE QUEDA
Swap — 4 GB	Colchón adicional para picos de memoria sin que el kernel mate procesos (OOM). Vive en el disco persistente (<code>/mnt/sda</code>), nunca en la raíz (que es un <code>tmpfs</code> en RAM — poner swap ahí sería contraproducente).	<code>/mnt/sda/swapfile</code> , prioridad <code>-2</code> , persistente en <code>/etc/fstab</code>
TCP BBR	Algoritmo de control de congestión más moderno que el default (<code>cubic</code>) — mejora el throughput/latencia percibida en conexiones con algo de pérdida de paquetes o distancia geográfica.	<code>/etc/sysctl.d/99-bbr.conf</code> (<code>net.ipv4.tcp_congestion_control=bbr</code> , <code>net.core.default_qdisc=fq</code>)
Límite de file descriptors	Sube el tope de conexiones/archivos abiertos simultáneos que puede manejar un proceso, de un default bajo a 65535.	<code>/etc/security/limits.conf</code>

PARA EL INGENIERO

Todo esto queda reproducible en un solo script idempotente:

```
bash infra/proxy/bootstrap-gcp.sh # aplica todo:
swap+BBR+limits+Caddy+monitor
bash infra/proxy/bootstrap-gcp.sh --skip-os-tuning # solo Caddy + monitor
bash infra/proxy/bootstrap-gcp.sh --no-monitors # sin (re)lanzar el monitor de RAM
```

Verificado en vivo, dos corridas seguidas contra el servidor real: la primera aplicó lo que faltaba, la segunda detectó todo ya configurado y no tocó nada — el paso de Caddy siempre valida antes de aplicar y recarga sin cortar servicio.

INVARIANTE

El monitor de RAM arranca en segundo plano (no como servicio del sistema) — **no sobrevive un reinicio de la VM**. Si la máquina se reinicia, hay que volver a correr `bootstrap-gcp.sh` para relanzarlo.

12. Dominios y rutas

DOMINIO	QUÉ SIRVE
<code>fluency.lat</code> , <code>www.fluency.lat</code>	Producción, detrás de Cloudflare (proxy naranja)
<code>qa.fluency.lat</code>	Pre-producción, DNS-only, directo al proxy real de GCP, sin CDN
<code>theruby.lat</code> y subdominios	Portfolio personal — fuera del producto Fluency, comparte el mismo Caddy

RUTA FRONTEND	QUIÉN LA VE	QUÉ MUESTRA
<code>/</code>	Público	Landing (marketing) o el módulo por defecto si no hay landing activo
<code>/pricing</code> , <code>/checkout</code>	Público	Planes y checkout
<code>/login</code>	Público	Login con Google
<code>/dashboard</code>	Autenticado	Home del dashboard (hub tras login)
<code>/flashcard</code>	Autenticado	Módulo de flashcards
<code>/pronoun-practice</code>	Autenticado	Práctica guiada de pronombres

13. Mapa de directorios del repositorio

Esta es la carpeta raíz del proyecto tal como existe en el disco de desarrollo (bajo perfil sparse `flashcards` — algunos módulos como `pronoun` o `admin` existen en la arquitectura pero no están materializados en este disco; ver §5.3).

CARPETA RAÍZ	PARA QUÉ SIRVE, EN UNA FRASE
<code>backend/</code>	Todo el código del servidor (API en Rust)
<code>client/</code>	Todo el código de la interfaz web (React)
<code>infra/proxy/</code>	Configuración de Caddy y scripts que instalan/actualizan los servidores
<code>json/</code>	El contenido real de las flashcards (frases, traducciones) en JSON, organizado por dirección de curso (<code>es_en</code> , <code>en_es</code> , etc.)
<code>card_images/</code> , <code>card_audio/</code>	Copia local de las imágenes/audio generados — en producción, el equivalente vive en el disco del proxy real
<code>docs/</code>	Toda la documentación técnica del proyecto, organizada por tema (arquitectura, módulos, infraestructura)
<code>modules/</code>	El "índice" legible por humanos de qué módulos de negocio existen y cómo se activan
<code>scripts/</code>	Herramientas de línea de comandos para operar el repo: cambiar perfil sparse, limpiar builds viejos de Azure, generar/auditar contenido
<code>tools/</code>	Herramientas de soporte que no son parte del producto: monitoreo en vivo de servidores, archivo histórico de infraestructura previa
<code>img/</code>	Utilidades de compresión/conversión de imágenes del propio repositorio (banner, scripts) — no confundir con <code>card_images/</code>
<code>keys/</code>	Llave SSH (<code>.pem</code>) usada para acceder a la instancia de AWS
<code>surreal_data/</code>	Datos persistidos por la instancia local de SurrealDB cuando se desarrolla en el PC

14. Operación del día a día (comandos)

NECESITO...	COMANDO
Levantar todo el entorno local (DB + IA + backend + Vite)	<code>./start.sh</code>
Levantar en modo lectura contra el catálogo real de producción	<code>./start.sh oracle</code> (nombre legado, hoy apunta a GCP)
Ver qué módulos están en disco ahora mismo	<code>./scripts/sparse-module.sh status</code>
Listar todos los módulos posibles	<code>./scripts/sparse-module.sh list</code>
Confirmar que producción responde	<code>curl -s https://fluency.lat/api/health</code>
Confirmar que QA responde	<code>curl -s https://qa.fluency.lat/api/health</code>
Compilar el backend solo con un módulo	<code>cargo build -p api_main --no-default-features --features auth,flashcards</code>
Verificar que toda la documentación de rutas del backend está al día	<code>./scripts/verify-blueprints.sh</code>
Reaplicar el tuning del proxy de GCP (swap, BBR, límites, Caddy, monitor)	<code>bash infra/proxy/bootstrap-gcp.sh</code>
Lanzar el pipeline manualmente	<code>az pipelines build queue --organization ... --definition-name "jcoronado1982.fluency" --branch main</code>
Limpiar historial/artefactos viejos de Azure DevOps	<code>./scripts/cleanup-ado-builds.sh --dry-run</code> (siempre simular primero)

REGLA DEL REPOSITORIO

Ningún comando que borre, pode o sobrescriba archivos (`rm` , `git clean` , `git restore` , cambio de perfil sparse) se ejecuta sin autorización explícita del operador en el momento. Documentado como regla absoluta en `CLAUDE.md` .

15. Glosario para quien no tiene experiencia en infraestructura

TÉRMINO	QUÉ SIGNIFICA AQUÍ
VM / instancia	Una "computadora virtual" alquilada a un proveedor de nube (Google, AWS, Azure). Tiene su propia RAM, CPU y disco, aislada de las demás.
Docker / contenedor	Una forma de empaquetar un programa junto con todo lo que necesita para correr, de modo que se comporte igual sin importar en qué máquina se instale.
VPC / red privada	Una red interna del proveedor de nube que conecta VMs entre sí sin pasar por internet pública — más rápida y más segura.
CDN (Content Delivery Network)	Una red de servidores repartidos geográficamente (Cloudflare, en este caso) que guarda copias de archivos "estáticos" cerca del usuario final para que carguen más rápido.
DNS	El sistema que traduce un nombre como <code>fluency.lat</code> a una dirección IP real.
SSL/TLS ("el candadito")	El cifrado que protege el tráfico HTTPS entre el navegador y el servidor. Caddy lo gestiona automáticamente con certificados de Let's Encrypt.
API / API REST	La "puerta" por la que el frontend le pide datos al backend, usando HTTP y JSON.
SPA (Single Page Application)	Una web que carga una sola vez y luego actualiza el contenido sin recargar la página completa — así funciona el frontend de React.
Arquitectura hexagonal / Clean Architecture	Un patrón de organización de código donde la lógica de negocio no depende directamente de una tecnología concreta, sino de contratos — así se puede cambiar de base de datos o proveedor de IA sin reescribir la lógica.
Monolito modular	Un solo programa (no microservicios separados) que internamente está dividido en módulos que se pueden activar/desactivar.
CI/CD	Automatizar la compilación y el despliegue del código cada vez que cambia, sin pasos manuales repetitivos.
Sparse-checkout	Función de Git para descargar solo una parte de los archivos de un repositorio.
RocksDB	Un motor de almacenamiento en disco (una "librería" de bajo nivel) que SurrealDB usa por dentro para guardar los datos.
Overflow / scale-to-zero	Capacidad extra que se enciende sola cuando hace falta (por ejemplo Cloud Run) y se apaga sola cuando no hay tráfico, para no pagar de más.
Swap	Espacio de disco que el sistema operativo usa como "RAM de emergencia" cuando la memoria física se agota — más lento que la RAM real, pero evita que el sistema mate procesos de golpe.
TCP BBR	Un algoritmo del kernel que decide qué tan rápido enviar datos por una conexión de red, más moderno y eficiente que el default en conexiones con distancia/pérdida de paquetes.
Espejo / mirror	Una copia de respaldo de la infraestructura, lista para asumir tráfico si el nodo principal falla.

16. Dónde seguir leyendo (documentos vivos)

Este PDF es una fotografía explicativa tomada el 4 de agosto de 2026. Para datos que cambian con el tiempo (IPs, límites de RAM, estado de un servicio, qué está habilitado o deshabilitado en el pipeline) la fuente de verdad siempre es el repositorio, no este documento:

TEMA	DOCUMENTO CANÓNICO EN EL REPO
Punto de entrada de toda la documentación	CLAUDE.md
Arquitectura de código (backend + frontend + registry + sparse)	docs/ARQUITECTURA_MODULAR.md
IPs, RAM, CPU, proveedor, contenedores activos	docs/infrastructure/server_inventory.md
Reglas operativas y presupuesto de recursos antes de optimizar	docs/infrastructure/AI_OPERATIONS_CONTEXT.md
Pipeline CI/CD paso a paso	docs/infrastructure/pipeline-and-deploy.md
Entrega y caché de imágenes/audio	docs/infrastructure/media-delivery-cache.md
Esquema de base de datos	database_schema_diagram.md
Dominios y rutas	docs/MAPA_DOMINIOS.md
Documentación por módulo de negocio	docs/modules/<módulo>.md
Reproducir el tuning del proxy de GCP	infra/proxy/bootstrap-gcp.sh
Credenciales y accesos	SECRETS_MAP.md (solo local, nunca en el repo público)

Fluency — documento generado a partir de la documentación viva del repositorio el 4 de agosto de 2026. No editar este PDF directamente: si algo cambia, se actualiza primero el documento canónico correspondiente en docs/ y se regenera este PDF a partir de ahí.